

LABORATORIO 07

¿Y comparado con qué?

Pruebas de rendimiento.

Guía para hacerlo por tu cuenta, paso a paso

Confluent Platform 8.2.0 · Kafka 4.2 en modo KRaft · 3 brokers

Duración estimada · 45 minutos, casi todos de leer lo que salió

NovaTech Logistics · caso de estudio

Antes de empezar

Qué vas a lograr

Al final de esta guía vas a haber medido el clúster, vas a haberlo medido otra vez **sin tocar nada**, y los dos números van a ser distintos. Todo lo demás del laboratorio sale de ahí.

Este es el laboratorio que enseña a **no creerle a un número**. No hay nada que configurar y nada que romper. El trabajo es leer.

LO QUE ESTE LABORATORIO DEMUESTRA

Una sola medición no dice nada. El rendimiento es siempre una comparación.

Y LA COMPARACIÓN TAMPOCO SE LEE SOLA

Una diferencia solo cuenta si es más grande que el ruido de tu máquina. Por eso lo primero que se mide no es la mejora, es el ruido.

UN AVISO, PORQUE ESTE LAB TERMINA DISTINTO A COMO ESPERAS

Al final del recorrido la respuesta medida va a ser «**en esta máquina no se distingue**».

No es que el laboratorio salga mal. **Es el resultado**, y aprender a decirlo en voz alta es la mitad de la clase.

Qué necesitas tener listo

Requisito	Cómo lo compruebas
Docker Desktop corriendo, con 6 GB de RAM	Ajustes de Docker Desktop, pestaña <i>Resources</i>
Git Bash abierto en la carpeta del lab	<code>cd Capitulo_3/lab-07-pruebas-rendimiento</code>
La máquina lo más descargada posible	Cierra el navegador con veinte pestañas y lo que esté sincronizando. Hoy estás midiendo tu propia máquina, y todo lo que corra ahí entra en la medición
Papel y lápiz, o un bloc de notas	Vas a anotar unos veinte números. No confíes en el desplazamiento de la terminal
Los puertos 9092, 9093, 9094 y 8090 libres	El paso 1 te avisa si no lo están

El caso

Viene la temporada alta y alguien pregunta cuánto aguanta el clúster de **NovaTech Logistics**. Se corre una prueba de carga y salen **114 000 mensajes por segundo**. Ese número entra en una lámina.

El equipo aplica un cambio de tuning que la documentación recomienda, vuelve a correr la prueba, y salen **111 000**. Conclusión de la reunión, *el cambio empeoró el clúster, vuélvano atrás*.

Las dos mediciones son reales. Y la conclusión **no se sostiene**. Pero cuidado, porque la contraria tampoco. Si los números hubieran salido al revés y la reunión hubiera decidido *el cambio funciona, déjenlo*, habrían estado **igual de equivocados**.

El problema no es el tuning. El problema es que **114 000 no significa nada por sí solo**, y nadie en esa reunión sabía cuánto se mueve ese número cuando no se cambia nada.

LA METÁFORA · HOY ENTRA EL AYUDANTE

Kafka sigue siendo **el restaurante**. El **mozo** es el broker, el **tipo de comanda** es el tópico, los **sectores del salón** son las particiones, el **cocinero** es el consumidor y la **brigada** es su grupo.

Hoy entra una pieza nueva, del otro lado del salón. **El ayudante**, que recoge las comandas de las mesas y se las lleva al mozo. Es el **productor**.

Todo lo que vas a tunear hoy son decisiones tuyas, no del mozo.

Las tres decisiones del ayudante

Decisión	El parámetro	Qué controla
Cuánto se queda esperando a que caigan más comandas antes de arrancar	<code>linger.ms</code>	Es el del recorrido de hoy
Cuántas comandas le caben en la bandeja	<code>batch.size</code>	Cuántos bytes junta antes de salir
A quién le espera el visto bueno	<code>acks</code>	A nadie, al mozo, o al mozo y a las libretas de respaldo

Un ayudante con `linger.ms=0` sale disparado apenas tiene algo en la mano. Si espera diez milisegundos más, sale con la bandeja más llena y hace menos viajes. Mueve más comandas por hora, y cada comanda individual llega un poco más tarde.

Y AQUÍ ESTÁ EL LABORATORIO DE HOY

Si cronometras **un** viaje del ayudante, no sabes nada. Ese viaje pudo tocar el ascensor libre o la cocina llena. Para decir algo hay que cronometrar varios, y hay que cronometrar los dos escenarios **el mismo día, uno detrás del otro**.

Levantar el clúster

Qué vamos a hacer

El mismo clúster de tres brokers de los laboratorios anteriores. Este lab trae además un tópico de banco de pruebas ya creado, con 6 particiones.

Los comandos

```
cd Capitulo_3/lab-07-pruebas-rendimiento
chmod +x bin/*.sh kafka-cli/*.sh
bin/start-lab.sh
```

`cd Capitulo_3/lab-07-pruebas-rendimiento` — la carpeta de este laboratorio. Todos los comandos salen de aquí

`chmod +x` — permiso de ejecución para los scripts, que en Git Bash a veces se pierde al clonar

`bin/start-lab.sh` — levanta los tres brokers, la interfaz web, y crea el tópico

`novatech.tuning.bench`

Comprueba que el banco de pruebas existe

```
kafka-cli/describe-topic.sh novatech.tuning.bench | head -1
```

LO QUE VAS A VER · RECORTADO A LO ANCHO

```
Topic: novatech.tuning.bench PartitionCount: 6 ReplicationFactor: 3
Configs: min.insync.replicas=2
```

VAS BIEN SI...

Salió ✓ 3/3 brokers operativos y el tópico `novatech.tuning.bench` dice `PartitionCount: 6`.

Seis particiones importan hoy, porque son las que permiten que la carga se reparta entre los tres brokers en vez de caer toda en uno.

SI EL TÓPICO NO EXISTE

El mensaje es `Topic 'novatech.tuning.bench' does not exist`. El `start-lab.sh` no llegó a crearlo.

Créalo tú mismo y sigue:

```
docker exec kafka-broker-1 kafka-topics \  
  --bootstrap-server kafka-broker-1:29092 \  
  --create --topic novatech.tuning.bench \  
  --partitions 6 --replication-factor 3
```

SI ALGO FALLA AL LEVANTAR

Es lo mismo de los laboratorios anteriores. **Docker apagado**, abre Docker Desktop. **Timeout: solo 2/3 brokers**, sube Docker a 6 GB y repite.

Y si dice `port is already allocated`, hay otro laboratorio del curso arriba que el `start-lab.sh` no supo bajar. **Los dos que se le escapan son `kafka-broker`, el del lab 01, y `kafka-broker-4`, el del lab 08**. Míralo y bájalo tú:

```
docker ps --filter name=kafka --format "{{.Names}}\t{{.Label  
  \"com.docker.compose.project\"}}\"  
docker compose -p novatech-labNN down -v --remove-orphans
```

Medir el ruido, que es lo primero

Qué vamos a hacer

Antes de tunear nada hay que saber **cuánto se mueve el número cuando no se cambia nada**. Ese movimiento es el ruido de tu máquina, y cualquier mejora más chica que ese ruido es indistinguible de la suerte.

Se mide dos veces seguidas, con el clúster en el mismo estado y sin tocar un solo parámetro.

CONCEPTO · THROUGHPUT

Cuánto trabajo pasa por unidad de tiempo. Aquí sale como `records/sec` y como `MB/sec`. **Es el número que la gente cita.**

CONCEPTO · LATENCIA

Cuánto tarda **un** mensaje en confirmarse. Aquí viene cinco veces, como promedio, como máximo, y en tres percentiles.

CONCEPTO · PERCENTIL

El `99th` es el valor que **el 99 % de los mensajes no superó**. Dicho al revés, uno de cada cien tardó **más** que eso.

El comando, dos veces seguidas

```
kafka-cli/perf-test.sh novatech.tuning.bench 50000
```

kafka-cli/perf-test.sh — envoltorio del curso. Por dentro llama a `kafka-producer-perf-test` dentro del contenedor

novatech.tuning.bench — a qué tópico se le manda la carga

50000 — cuántos mensajes generar. A 200 bytes cada uno son unos 10 MB

EL COMANDO REAL, EL QUE VAS A ESCRIBIR EN SUNAT

```
kafka-producer-perf-test \  
  --topic novatech.tuning.bench \  
  --num-records 50000 --record-size 200 --throughput -1 \  
  --producer-props bootstrap.servers=kafka-broker-1:29092 \  
    acks=all batch.size=16384 linger.ms=0 compression.type=none
```

--num-records 50000 — cuántos mensajes manda en total

--record-size 200 — cuántos bytes pesa cada uno

--throughput -1 — tope de mensajes por segundo. Con **-1** empuja todo lo que pueda, que es lo que queremos para medir el techo

--producer-props — las propiedades del productor. **Aquí vive el tuning**

acks=all — cuántas copias confirman antes de dar el mensaje por escrito

batch.size=16384 — 16 KB. Cuántos bytes junta antes de mandar un lote

linger.ms=0 — no espera nada, cierra el lote apenas puede. **Es el valor de fábrica, y el que vamos a mover en el paso 3**

compression.type=none — sin comprimir

LO QUE SALIÓ EN UNA CORRIDA REAL · DOS LÍNEAS, UNA POR CORRIDA

```
50000 records sent, 98231.827112 records/sec (18.74 MB/sec), 92.73 ms avg latency,  
190.00 ms max latency, 86 ms 50th, 166 ms 95th, 174 ms 99th, 177 ms 99.9th.
```

```
50000 records sent, 105708.245243 records/sec (20.16 MB/sec), 79.18 ms avg latency,  
196.00 ms max latency, 80 ms 50th, 120 ms 95th, 132 ms 99th, 138 ms 99.9th.
```

Cómo se lee

Medición	Corrida 1	Corrida 2	Diferencia
records/sec	98 232	105 708	+7,6 %
MB/sec	18,74	20,16	+7,6 %
Latencia media	92,73 ms	79,18 ms	-14,6 %
Latencia p99	174 ms	132 ms	-24,1 %

NO SE CAMBIÓ NADA ENTRE LAS DOS CORRIDAS

Mismo comando, mismo tópico, mismo clúster, unos segundos de diferencia. Y el throughput se movió un 7,6 %, la latencia media un 14,6 % y el p99 un 24,1 %.

Ese es el ruido de tu máquina, y acabas de medirlo. Cualquier mejora que anuncies por debajo de ese margen no es una mejora. Es la corrida siguiente.

VAS BIEN SI...

Las dos corridas no coinciden. Eso es todo lo que tiene que pasar en este paso.

Anota tu porcentaje. Es la vara con la que se mide todo lo que viene después, y en el paso 6 la vas a volver a mirar. **A ti te va a salir otro número**, y probablemente otra diferencia.

SI SALE `OPTION --PRODUCER-PROPS HAS BEEN DEPRECATED`

Es Kafka 8.x avisando de un cambio de nombre del parámetro. **El parámetro funciona.** Ignóralo, va a salir en todas las corridas del laboratorio.

SI TUS NÚMEROS SON MUCHO MÁS BAJOS QUE ESTOS

Por ejemplo 15 000 en vez de 100 000 `records/sec`. **Es normal y no rompe nada.** Estos números salieron de una máquina concreta, y el tuyo es otro.

Docker Desktop sobre Windows mete una capa de virtualización que se nota mucho aquí. **Da igual**, porque este laboratorio no compara tu máquina contra la mía. Compara tus corridas contra tus corridas.

SI LAS DOS CORRIDAS TE SALEN CASI IDÉNTICAS, CON UN 1 O 2 % DE DIFERENCIA

Es una máquina muy descargada, y **hay que tener cuidado con eso**. Con un ruido tan chico, ganancias del 3 % sí lo superarían.

Corre la base tres o cuatro veces más. El ruido casi siempre es mayor de lo que muestran dos corridas, y el paso 6 de esta guía trata exactamente de eso.

Cambiar un parámetro. Uno solo

Qué vamos a hacer

Ahora sí se tunea. Subimos `linger.ms` de 0 a 10. El ayudante, en vez de salir disparado con lo que tenga en la mano, se queda hasta diez milisegundos juntando comandas antes de arrancar. Menos viajes, cada uno más lleno.

SOLO ESE

`batch.size` se queda en 16 KB, `acks` en `all`, la compresión en `none`.

Si mueves tres cosas y el número mejora, tienes una mejora que **no sabes atribuir**. Y el día que una de las tres te muerda en producción, no vas a saber cuál sacar.

Antes de correr, apuesta

Escríbelo en tu papel antes de teclear. Al subir `linger.ms` de 0 a 10, **¿el throughput sube o baja? ¿Y la latencia media?** Comparar tu apuesta con el resultado es la mitad del ejercicio.

El comando, otra vez dos veces

```
kafka-cli/perf-test.sh novatech.tuning.bench 50000 --linger-ms 10
```

--linger-ms 10 — cuántos milisegundos espera el productor juntando mensajes antes de mandar el lote, aunque no esté lleno. **Es el único cambio respecto del paso 2**

LO QUE SALIÓ EN LA CORRIDA REAL

```
50000 records sent, 89605.734767 records/sec (17.09 MB/sec), 95.03 ms avg latency,  
235.00 ms max latency, 102 ms 50th, 150 ms 95th, 160 ms 99th, 164 ms 99.9th.
```

```
50000 records sent, 116279.069767 records/sec (22.18 MB/sec), 58.52 ms avg latency,  
199.00 ms max latency, 61 ms 50th, 89 ms 95th, 92 ms 99th, 96 ms 99.9th.
```

VAS BIEN SI...

Tienes cuatro corridas anotadas. **Todavía no las interpretas.**

Fíjate solo en una cosa. **Las dos corridas con `linger.ms=10` tampoco coinciden entre sí,** y se separan más que las dos del paso 2. El ruido no desaparece porque cambies un parámetro.

Un par más, y las dos lecturas de la tabla

Qué vamos a hacer

Con cuatro corridas todavía no se puede concluir nada, y este paso empieza mostrando por qué. Después agregamos un par más, medido uno detrás del otro.

Primero, mira lo que ya tienes

```
kafka-cli/perf-test.sh novatech.tuning.bench 50000
kafka-cli/perf-test.sh novatech.tuning.bench 50000 --linger-ms 10
```

los dos, uno detrás del otro — eso es un par. Se miden con la máquina en el mismo estado, con segundos de diferencia, para que lo único que cambie entre los dos sea el parámetro

EL TERCER PAR, MEDIDO

```
50000 records sent, 114942.528736 records/sec (21.92 MB/sec), 65.29 ms avg latency,
195.00 ms max latency, 71 ms 50th, 95 ms 95th, 99 ms 99th, 102 ms 99.9th.
```

```
50000 records sent, 124378.109453 records/sec (23.72 MB/sec), 54.47 ms avg latency,
194.00 ms max latency, 58 ms 50th, 79 ms 95th, 82 ms 99th, 84 ms 99.9th.
```

Las seis corridas, juntas

	records/sec	Latencia media	Latencia p99	Latencia máx
Base 1	98 232	92,73 ms	174 ms	190 ms
linger 1	89 606	95,03 ms	160 ms	235 ms
Base 2	105 708	79,18 ms	132 ms	196 ms
linger 2	116 279	58,52 ms	92 ms	199 ms
Base 3	114 943	65,29 ms	99 ms	195 ms
linger 3	124 378	54,47 ms	82 ms	194 ms

Lectura uno · los rangos sueltos no separan, y engañan en las dos direcciones

base 98 232 ————— 114 943
linger 89 606 ————— 124 378 se pisan

La **mejor** corrida base (114 943) le gana a la **peor** corrida tuneada (89 606). Un alumno que corriera esas dos se iría con «el tuning empeoró el clúster», que es exactamente la reunión del caso. Otro que corriera la base 1 contra la tuneada 3 se iría con «mejoró un **27 %**».

Las dos conclusiones saldrían de datos reales y las dos estarían mal.

Lectura dos · pareado se ve mucho mejor

Par	Base	<code>linger.ms=10</code>	Diferencia	¿Quién ganó?
1	98 232	89 606	-8,8 %	base
2	105 708	116 279	+10,0 %	<code>linger</code>
3	114 943	124 378	+8,2 %	<code>linger</code>

Dos de tres para el tuneado, y las dos ganancias superan el 7,6 % de ruido que mediste en el paso 2. Es el momento exacto en que se escribe la lámina que dice «`linger.ms=10` mejora el throughput un 10 %».

NO CIERRES AQUÍ. CON TRES PARES NO ALCANZA

Hay un par donde el tuneado **pierde**, y pierde por un 8,8 %, que es del mismo tamaño que las victorias.

Si un efecto es real, no se da vuelta de un par a otro. Lo que tienes por ahora es **una moneda que salió cara dos de tres veces**, y eso le pasa a cualquier moneda.

VAS BIEN SI...

Tienes los tres pares anotados con su porcentaje, y **comparaste cada uno contra el ruido del paso 2**.

A ti el marcador te puede salir 3-0, 0-3 o 1-2. Da igual cuál. Lo que se hace a continuación es lo mismo en todos los casos.

La tanda de control · diez pares

Qué vamos a hacer

Tres pares no contestan la pregunta. Vamos a correr diez, y con un detalle de diseño que importa.

POR QUÉ SE ALTERNA EL ORDEN DENTRO DE CADA PAR

Si una configuración corre **siempre** en la misma posición, cualquier cosa que dependa de la posición se le suma o se le resta siempre a la misma. Cachés que se calientan, otra tarea del sistema que arranca.

Alternar reparte ese efecto entre las dos.

Los comandos

```
# los pares impares · la base primero
kafka-cli/perf-test.sh novatech.tuning.bench 50000
kafka-cli/perf-test.sh novatech.tuning.bench 50000 --linger-ms 10

# los pares pares · el tuneado primero
kafka-cli/perf-test.sh novatech.tuning.bench 50000 --linger-ms 10
kafka-cli/perf-test.sh novatech.tuning.bench 50000
```

Son veinte corridas de unos cinco segundos cada una. **Toma unos dos minutos y no requiere que hagas nada más que esperar.**

Los diez pares medidos

Par	Orden	Base	linger.ms=10	Diferencia
1	base 1 ^o	118 483	124 069	+4,7 %
2	linger 1 ^o	123 153	124 069	+0,7 %
3	base 1 ^o	119 904	128 205	+6,9 %
4	linger 1 ^o	130 890	129 199	-1,3 %
5	base 1 ^o	125 000	129 870	+3,9 %
6	linger 1 ^o	99 602	126 263	+26,8 %
7	base 1 ^o	125 628	135 870	+8,2 %
8	linger 1 ^o	132 979	127 877	-3,8 %
9	base 1 ^o	125 628	128 205	+2,1 %
10	linger 1 ^o	121 951	120 482	-1,2 %

Resumen de los diez pares		Valor
Marcador		linger 7 · base 3
Media de la base		122 322 rec/s
Media del tuneado		127 411 rec/s
Diferencia de medias		+4,2 %
Rango de las diferencias por par		-3,8 % a +26,8 %

SIETE DE DIEZ. ESTE ES EL MOMENTO PELIGROSO DEL LABORATORIO

Ganar siete de diez **se siente como evidencia**. Es cuando se escribe la lámina y se manda el correo.

No la escribas todavía. Falta volver a mirar la vara, y la vara que tienes está mal medida. Eso es el paso 6.

VAS BIEN SI...

Tienes el marcador y las dos medias. **A ti te puede salir 5-5, 3-7 o 9-1.**

La conclusión de este laboratorio no depende de quién gane, y el paso siguiente explica por qué.

El ruido que dos corridas no te mostraron

Qué vamos a hacer

Este paso no necesita ningún comando nuevo. **Los datos ya los tienes.** Lo único que hay que hacer es mirarlos de otra manera.

En el paso 2 mediste el ruido con **dos** corridas base y te dio 7,6 %. En el paso 5 corriste **diez** corridas base más, sin tocar nada entre ellas. Ponlas en fila.

LAS DIEZ CORRIDAS BASE DE LA TANDA DE CONTROL, EN RECORDS/SEC

```
118483  123153  119904  130890  125000
 99602   125628  132979  125628  121951

mínimo   99 602
máximo  132 979
dispersión 33,5 %
```

AQUÍ SE CAE TODO LO ANTERIOR

El ruido real de esta máquina, medido sobre diez corridas sin tocar nada, es del **33,5 %**.

El 7,6 % del paso 2 no era el ruido. Era **lo que dos corridas alcanzaron a mostrar del ruido**, y se quedó cuatro veces corto.

Y ahora la comparación que cierra el laboratorio

```
ruido real, sin cambiar nada      |----- 33,5 % -----|
diferencia de medias del tuning  | 4,2 % |
la mejor ganancia de un par      |----- 26,8 % -----|
```

La diferencia de medias cabe ocho veces dentro del ruido. Y hasta el par que más ganó, ese +26,8 % que parecía espectacular, cabe entero dentro de lo que la máquina se mueve sola.

Mira además de dónde salió ese +26,8 %. La base de ese par fue **99 602**, la corrida más lenta de las diez. **No es que el tuneado corriera rápido. Es que la base corrió mal.**

En esta máquina, con esta carga, `linger.ms=10` no se distingue de la línea base.

POR QUÉ ESTO VALE MÁS QUE LA LÁMINA DEL 10 %

No es que `linger.ms=10` sea peor. Es que **el efecto, si existe, es más chico que lo que esta máquina se mueve sola**, y con estas herramientas no lo podemos medir.

«No se distingue» es un resultado, no un fracaso. Es lo que un ingeniero le lleva a su jefe cuando el jefe le pidió un número. *Medí, y la diferencia es más chica que mi error de medición. Si quieres que la persiga, necesito otro banco de pruebas.*

Esa frase vale más que la lámina del 10 %, porque la lámina del 10 % **se cae sola la primera vez que alguien la repite**.

LO LOGRASTE SI...

Puedes decir en una frase, sin mirar la guía, **por qué siete victorias de diez no alcanzan para afirmar que el tuning sirve**.

Y fíjate en lo que hizo falta para llegar ahí. **No una métrica distinta ni un comando más listo. Repetir hasta que el ruido dejara de mandar.**

SI TU GANANCIA MEDIA SÍ SUPERA TU RUIDO DE DIEZ CORRIDAS, Y POR MUCHO

Puede ser un efecto real en tu máquina, o tu máquina hizo algo raro. **Corre cinco pares más.**

Un efecto real se repite. Una corrida rara, no. Y si tras quince pares la ganancia sigue por encima del ruido, **tienes un resultado que puedes defender**, que es exactamente lo que este laboratorio te enseñó a construir.

El promedio esconde los casos malos

Qué vamos a hacer

Ahora no compares corridas. **Quédate dentro de una sola línea**, la primera del paso 2.

```
50000 records sent, 98231.827112 records/sec (18.74 MB/sec), 92.73 ms avg latency,  
190.00 ms max latency, 86 ms 50th, 166 ms 95th, 174 ms 99th, 177 ms 99.9th.
```

Métrica de la misma corrida	Valor
Latencia media	92,73 ms
Latencia p50	86 ms
Latencia p95	166 ms
Latencia p99	174 ms
Latencia máxima	190 ms

El promedio dice 93 ms. Pero uno de cada cien mensajes tardó más de 174, y hubo al menos uno que tardó **190 ms, el doble del promedio**.

Si tu tablero muestra el promedio, ese mensaje **no aparece en ninguna parte**. Y es exactamente el que el usuario nota, porque es el que se quedó esperando.

POR ESO EL RENDIMIENTO SE MIRA EN PERCENTILES

El promedio te dice cómo le fue al sistema. **El p99 te dice cómo le fue al peor de cada cien usuarios**. Los acuerdos de nivel de servicio se escriben con percentiles, nunca con promedios.

LO LOGRASTE SI...

Puedes señalar, en una sola línea de salida, **cuál es el número que se reporta y cuál es el que se vigila**. No son el mismo.

Lo que aprendiste

1 Mide dos veces antes de tunear una. Y después mide diez.

La primera medición no es la línea base, es una muestra. Y dos tampoco alcanzan. Lo viste en el paso 6, donde el ruido pasó de 7,6 % a 33,5 % con solo mirar más corridas. Todo lo que caiga dentro de ese rango es ruido.

2 Un parámetro a la vez.

Si cambias `linger.ms`, `batch.size` y la compresión juntos y mejora, tienes una mejora que no sabes atribuir. Sirve para la lámina y no sirve para operar.

3 Compara pares, no rangos, y mide los dos el mismo día.

Cada configuración contra la otra, una detrás de la otra, varias veces y alternando el orden.

Comparar la corrida de hoy contra un número anotado el mes pasado es comparar ruido. Y una mejora solo cuenta si es más grande que tu ruido, aunque gane siete pares de diez.

4 El promedio se reporta, el percentil se vigila.

El promedio esconde la cola. Si un tablero solo muestra promedios, los casos que el usuario nota son invisibles en él.

LA LECTURA QUE SE USA PARA TODO

Un número de rendimiento sin su comparación, su repetición y su percentil no es un dato. Es una anécdota.

Y su corolario, que es lo que separa a quien mide de quien presenta. **La respuesta «no se distingue» es una medición. La respuesta «mejoró un 10 %», dicha sin saber cuánto se mueve la máquina sola, no lo es.**

Para profundizar

Lo que sigue quedó fuera del recorrido por tiempo. **Todos estos comandos se ejecutaron contra este mismo clúster y las salidas son las que dieron.**

Y TODOS SE LEEN CON LA REGLA DEL RECORRIDO

Pares intercalados, varias veces, y cada diferencia se compara **contra el ruido**, no contra cero. **Los números de abajo son los de esta máquina.** Los tuyos van a ser otros, y puede que te lleven a otra conclusión. Lo que no cambia es el método.

A · Los tres niveles de `acks`

Es la decisión de a quién le espera el visto bueno el ayudante.

<code>acks</code>	Espera a	Qué pasa si el líder muere justo después de confirmar
0	Nadie	El mensaje se pierde y el productor nunca se entera
1	El líder	Se pierde si murió antes de que las réplicas copiaran
all	El líder y las réplicas en ISR	No se pierde

```
kafka-cli/perf-test.sh novatech.tuning.bench 50000 --acks 0
kafka-cli/perf-test.sh novatech.tuning.bench 50000 --acks 1
kafka-cli/perf-test.sh novatech.tuning.bench 50000 --acks all
```

TRES CORRIDAS DE CADA UNO, EN RECORDS/SEC

acks=0	142450	143266	147059	rango 142 450 a 147 059
acks=1	132626	136240	134771	rango 132 626 a 136 240
acks=all	126582	132275	130890	rango 126 582 a 132 275

Lo que hay que mirar. Aquí los tres rangos **no se pisan**, y el orden es el que dice el manual. Menos confirmaciones, más rápido. Es un efecto que se ve a simple vista, sin pares y sin estadística.

Y AUN ASÍ, CUIDADO CON LO QUE CONCLUYES

La diferencia entre `acks=0` y `acks=all` es de un 11 %, y tu ruido de diez corridas fue del 33,5 %. **Estos tres rangos no se pisan porque las nueve corridas cayeron juntas en el tiempo**, no porque el efecto sea mayor que el ruido del día.

En un clúster real, con los brokers en servidores distintos y switches en el medio, la diferencia es mucho mayor. **Aquí los tres brokers viven en la misma máquina**, así que esperar la confirmación de las réplicas no cruza una red de verdad.

LA REGLA QUE VALE MÁS QUE EL NÚMERO

Un banco de pruebas que no se parece a producción mide el banco de pruebas.

Y lo que `acks` decide de verdad no es la velocidad. **Es qué se pierde cuando un broker se cae**. Eso se elige por riesgo, no por benchmark.

B • `batch.size`, el otro parámetro del lote

`batch.size` es cuántos **bytes** junta el productor antes de mandar el lote. `linger.ms` es cuánto **tiempo** espera. Se confunden todo el tiempo.

```
kafka-cli/perf-test.sh novatech.tuning.bench 50000 --batch-size 65536
```

TRES CORRIDAS DE CADA UNO

	records/sec	latencia media
base (16 KB)	128 866 a 135 501	37,33 a 46,42 ms
64 KB	149 254 a 159 744	6,02 a 10,24 ms

ESTE ES EL CONTRASTE QUE CIERRA EL LABORATORIO

`linger.ms` necesitó trece pares para responder «no sé». `batch.size` **no necesita ninguno**. Los dos rangos de throughput no se tocan, y los de latencia tampoco se acercan.

Así se ve un efecto de verdad. Cuando hay que pelear con el ruido para encontrar una mejora, casi siempre la mejora no está.

Y el extremo opuesto, que es el más instructivo de todos:

```
kafka-cli/perf-test.sh novatech.tuning.bench 50000 --batch-size 1024
```

CON UN LOTE DE 1 KB

	records/sec	latencia media
1 KB	29 240 a 31 807	646,28 a 717,24 ms

El throughput se desploma a **una cuarta parte** y la latencia media sube **más de quince veces**. El **batching no es un ajuste fino**. Es lo que sostiene el rendimiento.

C · Compresión

```
kafka-cli/perf-test.sh novatech.tuning.bench 50000 --compression lz4
kafka-cli/perf-test.sh novatech.tuning.bench 50000 --compression zstd
```

LATENCIA MEDIA, TRES CORRIDAS DE CADA UNO

lz4	41,16	35,98	35,06 ms	rango 35,06 a 41,16
zstd	7,87	13,81	6,17 ms	rango 6,17 a 13,81

Lo que hay que mirar. `lz4` es notablemente **estable**, con tres corridas muy juntas. `zstd` es bastante más rápido y también mucho más disperso, con una corrida que dobla a las otras dos.

LA TRAMPA DE MEDIR COMPRESIÓN

Con datos de relleno como los de esta prueba la compresión rinde muchísimo, porque son muy repetitivos. Con datos reales, que comprimen bastante peor, rinde mucho menos. **Nunca midas compresión con datos sintéticos y lleves ese número a una decisión.**

D · Combinar parámetros, y por qué se hace al final

```
kafka-cli/perf-test.sh novatech.tuning.bench 50000 --batch-size 65536 --linger-ms 10
```

LO QUE SALIÓ

```
50000 records sent, 165562.913907 records/sec (31.58 MB/sec), 8.87 ms avg latency,
196.00 ms max latency, 7 ms 50th, 26 ms 95th, 29 ms 99th, 31 ms 99.9th.
```

Es el mejor número de todo el laboratorio. **Y no te dice cuál de los dos parámetros lo consiguió.**

Lo que hay que mirar. Combinar solo tiene sentido **después** de haber medido cada parámetro por separado. Ya sabes que uno mueve el throughput con claridad y del otro no puedes afirmar nada. Si hubieras ido directo a la combinación, te llevarías el 165 000 sin saber que `linger.ms` no aportó nada demostrable.

E · El lado del consumidor, y una trampa fea

```
kafka-cli/consumer-perf-test.sh novatech.tuning.bench 100000
kafka-cli/consumer-perf-test.sh novatech.tuning.bench 100000 --fetch-size 5242880
```

SALIDA REAL, CON LAS COLUMNAS QUE IMPORTAN

```
data.consumed.in.MB, MB.sec, data.consumed.in.nMsg, nMsg.sec, rebalance.time.ms,
fetch.time.ms, fetch.MB.sec, fetch.nMsg.sec
19.1238, 5.1477, 100264, 26988.9637, 3574, 141, 135.6301, 711092.1986
```

MIRA LAS DOS COLUMNAS DEL MEDIO ANTES DE CREEERLE AL NMSG.SEC

`nMsg.sec` dice **26 989** mensajes por segundo. Pero de los ~3 715 ms que duró la prueba, **3 574** fueron el rebalanceo y solo **141** fueron leer.

El `nMsg.sec` está midiendo casi puro arranque. La columna honesta es la de al lado, `fetch.nMsg.sec`, que dio **711 092**. Veintiséis veces más.

Y por eso subir `--fetch-size` a 5 MB casi no mueve el `nMsg.sec`, que pasó a 29 386. No era el fetch el que mandaba.

Antes de tunear algo, confirma que ese algo es el cuello de botella.

F · Particionado · todo el tráfico en una partición

Producir con **una sola clave** manda todo a **una** partición, y una partición es un solo broker líder. Sin paralelismo.

Primero anota dónde está cada partición:

```
docker exec kafka-broker-1 kafka-get-offsets \
  --bootstrap-server kafka-broker-1:29092 \
  --topic novatech.tuning.bench --time -1
```

Ahora escribe 50 000 mensajes con clave:

```
seq 1 50000 | awk '{ print "K:msg_"$1 }' | \
  docker exec -i kafka-broker-1 kafka-console-producer \
    --bootstrap-server kafka-broker-1:29092 \
    --topic novatech.tuning.bench \
    --property "parse.key=true" --property "key.separator=:" \
    --producer-property "batch.size=65536" --producer-property "linger.ms=10"
```

seq 1 50000 — genera los números del 1 al 50 000, uno por línea

`awk '{ print "K:msg_"$1 }'` — convierte cada número en una línea `K:msg_N`

`key.separator=:` — corta en el **primer** dos puntos. **Ojo con lo que es la clave aquí, porque no es lo que parece.** De la línea `K:msg_1` la clave es `K` y el valor es `msg_1`. Las 50 000 líneas llevan la **misma** clave, y ese es todo el truco del ejercicio

LOS OFFSETS, ANTES Y DESPUÉS

antes	después
<code>novatech.tuning.bench:0:425294</code>	<code>novatech.tuning.bench:0:425294</code>
<code>novatech.tuning.bench:1:430598</code>	<code>novatech.tuning.bench:1:430598</code>
<code>novatech.tuning.bench:2:422439</code>	<code>novatech.tuning.bench:2:422439</code>
<code>novatech.tuning.bench:3:428054</code>	<code>novatech.tuning.bench:3:428054</code>
<code>novatech.tuning.bench:4:417758</code>	<code>novatech.tuning.bench:4:467758</code> ← +50 000
<code>novatech.tuning.bench:5:425857</code>	<code>novatech.tuning.bench:5:425857</code>

Cinco particiones no se movieron ni un offset. Las 50 000 cayeron enteras en la 4, porque el *hash* de `K` da esa y siempre va a dar esa. Seis particiones de capacidad, una trabajando.

LO QUE ESTE EJERCICIO NO MIDE

`kafka-console-producer` no imprime `records/sec`, así que **no hay número que comparar** con el del recorrido. Lo que demuestra es **dónde caen los mensajes**, y eso se mide con los offsets.

La pregunta que vale. Si seis vehículos VIP generan el 80 % del tráfico y particionas por clave, ¿qué le pasa a la partición que les tocó? Es el *hot partitioning*, y es el precio de garantizar orden por clave.

G · El reporte del laboratorio

`plantillas/reporte-entregable.md` recorre estas actividades con sus preguntas. Las respuestas de referencia están en `soluciones/reporte-resuelto.md`.

OJO CON EL DISCO SI HACES TODA ESTA SECCIÓN

Cada corrida escribe 50 000 mensajes de 200 bytes, o sea unos 10 MB, y el tópico tiene factor de replicación 3. **La cuenta es 30 MB por corrida.**

El recorrido son 26 corridas y esta sección agrega unas veinte más, así que **puedes acercarte al gigabyte y medio**. No es un problema en una máquina normal, pero si Docker Desktop te avisa de espacio, el que libera es `bin/reset-lab.sh`. `bin/stop-lab.sh` **no sirve para esto**, porque apaga los contenedores y conserva los volúmenes a propósito.

Antes de cerrar

DEJA EL TERRENO LIMPIO

Si vas a seguir con el laboratorio 08, **deja el clúster arriba**. Si terminas por hoy:

```
bin/stop-lab.sh
```

Y comprueba con `docker ps --filter name=kafka-broker` que no quede nada.

LO QUE TE LLEVAS

La próxima vez que alguien te muestre un número de rendimiento, la primera pregunta ya no es «¿es bueno?». Es **«¿comparado con qué, cuántas veces lo mediste, y cuánto se mueve solo?»**.

Y la segunda cosa, que cuesta más. **Este laboratorio terminó sin encontrar la mejora que fue a buscar, y eso está bien**. Medir en serio incluye poder decir «no se distingue». El que nunca reporta ese resultado no es que tenga mejores parámetros. Es que no está midiendo el ruido.